PL/SQL Training

SESSION 8: Dynamic SQL in PL/SQL

DAVA.X ACADEMY SESSIONS

Svetlana Sura & Iulian Aurel Cozma

10–20 JUNE 2025

endava

SESSION 8

Dynamic SQL in PL/SQL

- Overview of dynamic SQL
- Dynamic DDL
- Dynamic DML
- Dynamic Queries
- Dynamic PL/SQL
- Advanced topics
- Best Practices
- SYS_REFCURSOR
- REF CURSOR
- Cursor vs REF CURSOR vs SYS_REFCURSOR

What is Dynamic SQL in PL/SQL?

Dynamic SQL refers to SQL statements that are constructed and executed at runtime, rather than being hardcoded in the PL/SQL block.

This allows for flexibility, such as:

- Executing DDL statements (CREATE, ALTER, DROP)
- Building SQL statements conditionally
- Executing SQL with unknown table/column names at compile time

Syntax

Native Dynamic SQL (NDS) statements:

EXECUTE IMMEDIATE *SQL_string*
[**INTO** {*define_variable*[, *define_variable*]... | *record*}]
[**USING** [**IN** | **OUT** | **IN OUT**] *bind_argument*[, [**IN** | **OUT** | **IN OUT**] *bind_argument*]...];

SQL_string – Is a string expression containing the SQL statement or PL/SQL block.
define_variable – Is a variable that receives a column value returned by a query.
record – Is a record based on a user-defined TYPE or %ROWTYPE that receives an entire row returned by a query.
bind_argument – Is an expression whose value is passed to the SQL statement or PL/SQL block, or an identifier that serves as an input and/or output variable to the function or procedure that is called in the PL/SQL block.

Sample:

- EXECUTE IMMEDIATE dynamic_string;
- EXECUTE IMMEDIATE dynamic_string INTO variable;
- EXECUTE IMMEDIATE dynamic_string USING bind_variable;

Dynamic SQL & PL/SQL

OPEN *cursor_name* **FOR** *SQL_string*

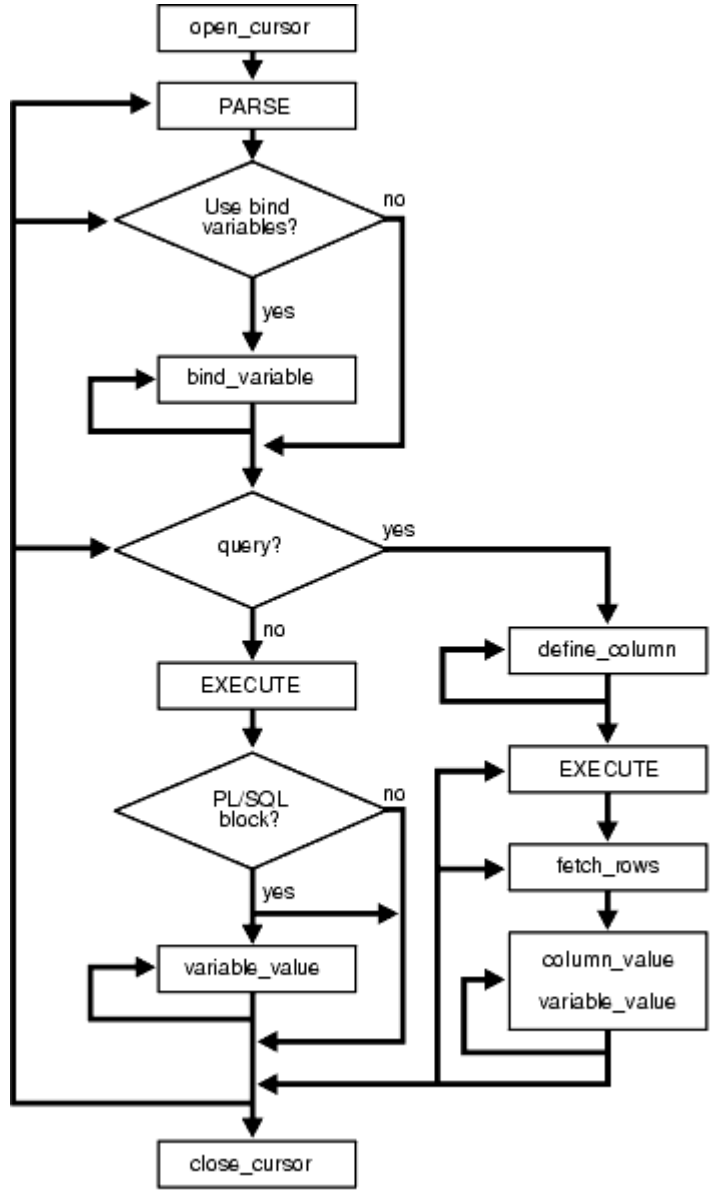
[**USING** [**IN** | **OUT** | **IN OUT**] *bind_argument*

[, [**IN** | **OUT** | **IN OUT**] *bind_argument*]...];

SQL_string – Contains the SELECT statement to be executed dynamically.

DBMS_SQL:

- *Parse Very Long Strings*
  - DBMS_SQL.PARSE overloading that supports VARCHAR2 (maximum bytes per line is 4000) or DBMS_SQL.VARCHAR2A (maximum bytes per line is 32767) or CLOB (4G-1)
  - **EXECUTE IMMEDIATE** is limited to 32K
- *Obtain Information About Query Columns*
  - DBMS_SQL allows you to describe the columns of your dynamic cursor, returning information about each column in an associative array of records. This capability offers the possibility of writing very generic cursor-processing code.



📌 Use Cases

endava

Use Case	Supported by Dynamic SQL?
Creating/dropping tables	✅ Yes
Changing column names	✅ Yes
Select into variable	✅ Yes
Insert/update/delete rows	✅ Yes
Parameterized queries	✅ Yes, with USING
Returning multiple rows	❌ Use OPEN FOR with cursors instead
Use Case	Supported by Dynamic SQL?

⚠️ Cautions with Dynamic SQL

- SQL Injection risk if you concatenate user input into the string.
- Always use USING.Harder to debug compared to static SQL.
- Slightly lower performance than static SQL due to runtime parsing.

✅ Best Practices

1. Use BIND VARIABLES with USING clause to improve security and performance.
2. Avoid using dynamic SQL when regular SQL would work.
3. Always handle exceptions like INVALID_CURSOR, TOO_MANY_ROWS, etc.

Safe Coding Tips for Dynamic SQL with Table Names

Technique	Purpose
DBMS_ASSERT.SQL_OBJECT_NAME	Validates object names (e.g. tables)
user_tables check	Verifies if table exists
Avoid user inputs directly	Always sanitize or validate first
Only expose whitelisted tables	For public/procedure APIs

❌ Example of What Not to Do:

endava

- **BAD: allows SQL injection**

```
v_sql := 'SELECT * FROM ' || p_table_name;
EXECUTE IMMEDIATE v_sql;
```

👎 **If p_table_name := 'DUAL WHERE 1=1; DROP TABLE employees; --', your data is gone.**

❗ Why the Injection Fails in Oracle EXECUTE IMMEDIATE

In Oracle PL/SQL, unlike some scripting languages, EXECUTE IMMEDIATE only accepts one SQL statement at a time.

```
CREATE OR REPLACE PROCEDURE vulnerable_proc(p_name IN VARCHAR2) IS
 v_sql VARCHAR2(1000);
BEGIN
 v_sql := 'SELECT COUNT(*) FROM sample WHERE name = ''' || p_name || '''';
 EXECUTE IMMEDIATE v_sql;
END;
```

That means this injected code: **vulnerable_proc('x''; DROP TABLE sample; --');**

leads to: *SELECT COUNT(*) FROM sample WHERE name = 'x'; DROP TABLE sample; --'*

Which Oracle does NOT allow — it raises:**ORA-00933: SQL command not properly ended**

✔ **Oracle prevents multiple statements from being executed in one EXECUTE IMMEDIATE, making it slightly safer by default — but not immune!**

Common Errors

endava

Error Code	Meaning
ORA-00900	Invalid SQL statement
ORA-00933	SQL command not properly ended
ORA-01008	Not all variables bound

REF CURSOR

IS REF CURSOR is used to define your own cursor variable types.

This gives you more control and allows for strong typing if needed.

There are tow type of ref cursor.

1. **Weak REF CURSOR** –it is flexible — the cursor can return any result set, but has no compile-time checking.
2. **Strong REF CURSOR** -is type-safe and helps catch mismatched queries at compile time.

Weak REF CURSOR Example

```
DECLARE
 TYPE weak_refcur IS REF CURSOR;
 c   weak_refcur;
 v_name employees.first_name%TYPE;
BEGIN
 OPEN c FOR SELECT first_name FROM employees WHERE department_id = 90;

 LOOP
  FETCH c INTO v_name;
  EXIT WHEN c%NOTFOUND;
  DBMS_OUTPUT.PUT_LINE(v_name);
 END LOOP;

 CLOSE c;
END;
```

Strong REF CURSOR Example

```
DECLARE
 TYPE emp_cur_type IS REF CURSOR RETURN employees%ROWTYPE;
 c   emp_cur_type;
 v_emp employees%ROWTYPE;
BEGIN
 OPEN c FOR SELECT * FROM employees WHERE department_id = 60;

 LOOP
  FETCH c INTO v_emp;
  EXIT WHEN c%NOTFOUND;
  DBMS_OUTPUT.PUT_LINE('Name: ' || v_emp.first_name || ' ' || v_emp.last_name);
 END LOOP;

 CLOSE c;
END;
```

When to Use IS REF CURSOR

Scenario	Use Strong Typing?	Use Weak Typing?
Known structure (e.g. employees%ROWTYPE)	✅ Yes	❌ Optional
Reusable for many different queries	❌ Not needed	✅ Yes
Returning cursor from procedure/function	✅ Either works	✅ Works well

🔍 What is SYS_REFCURSOR in Oracle?

endava

- SYS_REFCURSOR is a predefined weak REF CURSOR type.
- It is used to open a cursor dynamically and pass it around (e.g., from a procedure to a client).
- Unlike static cursors, it doesn't require a fixed return structure at compile time.

Using SYS_REFCURSOR in a Procedure (OUT parameter)

```
CREATE OR REPLACE PROCEDURE get_salaries(
    p_job_id IN VARCHAR2,
    p_cursor OUT SYS_REFCURSOR )
AS
BEGIN
  OPEN p_cursor FOR
            SELECT employee_id, salary
            FROM employees
            WHERE job_id = p_job_id; ....
END;
```

Call It Like This:

```
DECLARE
  c           SYS_REFCURSOR;
  v_id        employees.employee_id%TYPE;
  v_salary    employees.salary%TYPE;
BEGIN
 get_salaries('IT_PROG', c);

 LOOP
    FETCH c INTO v_id, v_salary;
    EXIT WHEN c%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE('ID: ' || v_id || ' | Salary: ' || v_salary);
 END LOOP;

 CLOSE c;
END;
```

⚙️ When to Use SYS_REFCURSOR

endava

Use Case	Should You Use SYS_REFCURSOR?
Return result sets from procedures	✅ Yes
Dynamic query output	✅ Yes
Performance-critical fixed structure	❌ Use static cursors
Need strict type checking at compile time	❌ Use strong typed cursors

Cursor vs REF CURSOR vs SYS_REFCURSOR

endava

Feature	Static Cursor (CURSOR)	REF CURSOR (IS REF CURSOR)	SYS_REFCURSOR
Definition	Named cursor with fixed query	Cursor variable (pointer to a result set)	Predefined weak REF CURSOR
Return type	Fixed at compile time	Strong or weak typing	Always weak typing
Flexibility	Low	High	High
Can be passed as parameter?	❌ No	✅ Yes	✅ Yes
Used with dynamic SQL?	❌ No	✅ Yes	✅ Yes
Can return different queries?	❌ No	✅ Yes (if weak typed)	✅ Yes
Common usage	Internal row-by-row fetch. Cannot be passed as OUT parameter	API result sets, dynamic procedures	Procedures/functions returning data
Compile-time type checking?	✅ Yes (strict)	✅ (if strong), ❌ (if weak)	❌ No

When to Use Which?

Use Case	Best Choice
Simple internal loop over query	CURSOR
Return result set to client	SYS_REFCURSOR
Pass structured data with checks	Strong REF CURSOR
Dynamic or variable SELECT queries	SYS_REFCURSOR or weak REF CURSOR

endava

SESSION 8

PL/SQL Data Dictionary

PL/SQL Data Dictionary

The PL/SQL Data Dictionary is a collection of read-only metadata tables and views that Oracle provides to give information about:

- Tables, views, columns
- Users, roles, privileges
- Indexes, constraints
- Procedures, packages, triggers
- And more…

It is essential for writing dynamic SQL, generating reports, validating object existence, and security audits.

📘 What Is the Oracle Data Dictionary?

endava

The data dictionary is a set of views and tables maintained by Oracle that store information about:

- Database structure (tables, views, indexes, triggers, etc.)
- Users and permissions
- PL/SQL objects (procedures, functions, packages, cursors)
- System statistics and configuration

It is **automatically updated by Oracle** and **read-only to users**.

Categories of Data Dictionary Views

Prefix	Description
USER_	Objects **owned by the current user**
ALL_	Objects the user **has access to**, even if not owner
DBA_	**All objects in the database**, only for DBA roles
V$	**Dynamic performance views** (memory, sessions, stats)
GV$	Global (RAC) version of V$ views

Common Data Dictionary Views for PL/SQL Development

🔧 PL/SQL Programs

endava

View	Description
USER_OBJECTS	All PL/SQL objects created by the user
USER_PROCEDURES	Info about functions/procedures owned by user
USER_SOURCE	The actual source code for PL/SQL objects
USER_ERRORS	Compilation errors in PL/SQL code

📊 Tables and Columns

endava

View	Description
USER_TABLES	Tables you own
USER_TAB_COLUMNS	Columns in your tables
USER_CONSTRAINTS	Constraints (PK, FK, CHECK, UNIQUE)
USER_INDEXES	Index info

Security and Grants

View	Description
USER_ROLE_PRIVS	Roles granted to the user
USER_SYS_PRIVS	System privileges granted
USER_TAB_PRIVS	Table-level grants

Good to Know

🧠 Why Use the Data Dictionary?

1. **Debugging**: View compilation errors (USER_ERRORS)
2. **Dynamic SQL**: Validate objects before building queries
3. **Code introspection**: Generate documentation or object maps
4. **Security**: Audit privileges and ownership

🔐 **Best Practice:**

Use USER_ views for current user development, and switch to ALL_ or DBA_ views only when:

- You're managing cross-schema access
- You’re a DBA doing full database reporting

Summary & Homework

THEORY

PL/SQL 101 Part 10, 12

PL/SQL Tutorial

PRACTICE

1. **DYN1: Introduction to Dynamic SQL in PL/SQL (PL/SQL Channel)**
2. **DYN3: Method 2 Dynamic SQL - Non-query DML with bind variables**

HOMEWORK

1. **Quiz on OPEN FOR with Dynamic Query**
2. **Quiz on Declaring a weak REF CURSOR type**
3. **Declaring REF CURSOR Types and Cursor Variables**

Takeaway